■ COMPLETE NOTES — TOPIC 03

CSS Selectors

Basic se Advanced — Poora Selector Guide

Web Development Course – Shauray

Universal Selector | Element Selector | Class Selector

ID Selector | Group Selector | Descendant

Child > | Adjacent + | Sibling ~

Attribute Selectors | Pseudo-Classes | Pseudo-Elements

Specificity | :is() :where() :not()

// TOPIC 01 — BASIC

Basic Selectors

Universal, Element, Class, ID aur Group — CSS ka foundation

KYA HAI?

CSS Selector ek **pattern** hai jo batata hai ki kaunse HTML element(s) pe CSS rule apply hogi. Selector CSS ka sabse important part hai — sahi selector pata hoga toh exactly wahi element style hoga jo chahiye. Selectors basic se advanced tak hote hain.

1. Universal Selector — *

universal.css

```css
/* * — Page ke SAARE elements select karta hai */
* {
    margin:     0;
    padding:    0;
    box-sizing: border-box;
}

/* Specific element ke andar ke saare elements */
.card * {
    color: white;   /* card ke andar sab white */
}
```

Warning: `*` ko sirf CSS reset ke liye use karo — baki jagah performance affect ho sakti hai kyunki har element ko style apply karta hai!

2. Element / Type Selector

element.css

```css
/* HTML tag ka naam likhdo — saare waise elements style honge */
h1     { font-size:   2.5rem;  color: white;    }
p      { line-height: 1.7;     color: #a8b2d8; }
a      { color:       #00b4d8; text-decoration: none; }
button { cursor:      pointer; }
img    { max-width:   100%;    height: auto;    }
```

3. Class Selector — .

class.css

```css
/* .classname — class attribute wale saare elements */
.btn     { background: #e94560; color: white; padding: 10px 20px; }
.card    { border: 1px solid #1e3a5f; border-radius: 8px; }
.active  { border-color: #06d6a0; }

/* Ek element pe multiple classes */
<div class="card active featured">...</div>

/* Specific element + specific class */
p.highlight  { background: yellow; }  /* sirf p jo highlight class hai */
div.card     { padding: 20px; }      /* sirf div jo card class hai */

/* Do classes dono saath ho toh (no space) */
.btn.active  { opacity: 1; }   /* btn AND active dono class ho toh */
```

4. ID Selector —

id.css

```css
/* #idname — sirf ek unique element per page */
#navbar  { background: #0f3460; height: 60px; }
#hero    { min-height: 100vh; }
#footer  { background: #071120; }
```

5. Group Selector — , (Comma)

group.css

```css
/* Comma se multiple selectors ek saath */
h1, h2, h3, h4 {
    font-family: 'Syne', sans-serif;
    color:       white;
    font-weight: 700;
}

.btn, .link, a {
    cursor: pointer;
    text-decoration: none;
}
```

Selector	Syntax	Kya Select Karta Hai	Specificity
Universal	*	Saare elements	0,0,0
Element	p, h1, div	Us type ke saare tags	0,0,1
Class	.btn, .card	Us class wale saare elements	0,1,0
ID	#navbar, #hero	Us ID wala ek element	1,0,0
Group	h1, h2, p	Saare listed selectors	Each apni

■ **Tip:** Real projects mein zyada se zyada **class selectors** use karo — ID selectors sirf JavaScript targeting aur anchor links ke liye rakho. Classes reusable hoti hain, IDs nahi!

// TOPIC 02 — INTERMEDIATE

Combinator Selectors

Descendant, Child, Adjacent, Sibling — relationships se select karo

KYA HAI?

Combinator selectors do elements ke beech ki **relationship** ke basis pe select karte hain. Yeh batate hain ki element kahan hai — kisi ke andar, kisi ke theek baad, ya kisi ka sibling. Inhe samajhna bahut zaroori hai precise targeting ke liye.

1. Descendant Selector — (space)

descendant.css

```css
/* A B — A ke andar ke SAARE B (kitne bhi deep ho) */
nav a {
    color: white;
    text-decoration: none;
}
/* nav ke andar ke saare <a> — chahe kitne bhi nested ho */

.card p   { color:    #a8b2d8; }
.card img { width:    100%;    }
main h2   { font-size: 1.8rem; }

/* HTML structure: */
<nav>                    ← parent
  <ul>
    <li><a>Home</a></li>  ← yeh bhi select hoga
    <li><a>About</a></li> ← yeh bhi
```

```
  </ul>
</nav>
```

2. Child Selector — >

child.css

```
/* A > B — sirf A ke DIRECT children B */
ul > li {
    list-style: none;
    padding:    8px 0;
}
/* Sirf ul ke direct li children — nested ul ke li nahi */

.nav > a      { font-weight: 600;  }
.wrapper > div{ padding:     20px; }

/* HTML: */
<ul>                        ← parent
  <li>Item 1</li>           ←   select hoga (direct child)
  <li>                      ←   select hoga
    <ul>
      <li>Nested</li>       ←   select NAHI hoga (grandchild)
    </ul>
  </li>
</ul>
```

3. Adjacent Sibling — +

adjacent.css

```
/* A + B — A ke THEEK BAAD wala B (sirf ek) */
h2 + p {
    font-size:   1.1rem;
```

```
    color:       #e8eaf6;
    font-weight: 500;   /* h2 ke baad ka pehla p thoda bada */
}

label + input {
    margin-top: 6px;
}

/* HTML: */
<h2>Heading</h2>
<p>Yeh select hoga  </p>   ← h2 ke theek baad
<p>Yeh NAHI hoga  </p>    ← theek baad nahi, skip ho gaya
```

4. General Sibling — ~

sibling.css

```
/* A ~ B — A ke baad ke SAARE B siblings */
h2 ~ p {
    color: #a8b2d8;
}
/* h2 ke baad ke saare p tags — chahe kitne bhi ho */

/* HTML: */
<h2>Heading</h2>
<p>Pehla para  </p>    ← select hoga
<div>Ek div</div>
<p>Doosra para  </p>   ← yeh bhi select hoga
<p>Teesra para  </p>   ← yeh bhi!
```

Combinator	Syntax	Kya Select Karta Hai
Descendant	A B (space)	A ke andar ke SAARE B (kitne bhi deep)
Child	A > B	A ke sirf DIRECT children B

Combinator	Syntax	Kya Select Karta Hai
Adjacent	A + B	A ke THEEK BAAD ka ek B sibling
General Sibling	A ~ B	A ke baad ke SAARE B siblings

■ **Tip:** `h2 + p` bahut common use case hai — jab heading ke theek baad wale paragraph ko special style dena ho, jaise intro text thoda bada karna!

// TOPIC 03 — INTERMEDIATE

Attribute Selectors

HTML attributes ke basis pe elements select karo

KYA HAI?

Attribute selectors kisi element ke **HTML attribute aur uski value** ke basis pe style karte hain. Yeh bahut powerful hain — bina class lage bhi elements precisely target kar sakte ho. Square brackets `[ ]` mein likhte hain.

attribute-selectors.css

```css
/* [attr] — woh elements jinke paas yeh attribute hai */
input[required] {
    border-color: #e94560;   /* required fields red border */
}
a[target] { color: orange; }

/* [attr="value"] — exact value match */
input[type="text"]     { border: 2px solid #00b4d8; }
input[type="email"]    { border: 2px solid #06d6a0; }
input[type="password"] { border: 2px solid #ffd166; }
a[target="_blank"]     { color:  #9d4edd;         }

/* [attr~="value"] — space-separated list mein value ho */
[class~="featured"] {
    border: 2px solid gold;
}
/* class="card featured active" wale elements select honge */

/* [attr^="value"] — value se SHURU hota ho (starts with) */
a[href^="https"] {
    color: #06d6a0;   /* https links green */
```

```
}
a[href^="http"]  { color: orange; }  /* http (non-secure) orange */
a[href^="mailto"]{ color: #ffd166;}  /* email links yellow */
a[href^="#"]     { color: #9d4edd;}  /* anchor links purple */

/* [attr$="value"] — value pe KHATAM hota ho (ends with) */
a[href$=".pdf"]  { color: red;   }  /* PDF links */
a[href$=".zip"]  { color: blue;  }  /* ZIP download links */
img[src$=".webp"]{ border: none; }

/* [attr*="value"] — value kahin bhi ho (contains) */
a[href*="youtube"] {
    color: #e94560;   /* youtube wale links red */
}
[class*="btn"] {
    cursor: pointer;   /* btn, btn-primary, btn-sm sab */
}

/* Case-insensitive match — i flag */
input[type="TEXT" i] { color: white; }
/* "text", "TEXT", "Text" — sab match karta hai */
```

Syntax	Matlab	Example
[attr]	Attribute exist karta ho	input[required]
[attr="val"]	Exact match	input[type="email"]
[attr~="val"]	Space list mein ho	[class~="active"]
[attr^="val"]	Se shuru ho (starts with)	a[href^="https"]
[attr$="val"]	Pe khatam ho (ends with)	a[href$=".pdf"]
[attr*="val"]	Kahin bhi ho (contains)	a[href*="google"]
[attr="val" i]	Case-insensitive match	input[type="TEXT" i]

■ **Tip:** `a[href^="https"]` se saare secure links ko style kar sakte ho — aur `a[href$=".pdf"]` se PDF links ke saamne automatically icon laga sakte ho `content` property se!

// TOPIC 04 — ADVANCED

Pseudo-Classes

Element ki state ya position ke basis pe style karo — : (single colon)

KYA HAI?

Pseudo-class ek keyword hai jo element ki **special state** define karta hai — jaise hover karna, focus hona, ya list mein first child hona. Ek colon `:` se likhte hain. Inse interactive aur dynamic styling hoti hai bina JavaScript ke!

User Action Pseudo-Classes

action-pseudo.css

```css
/* :hover — mouse upar aaye tab */
.btn:hover {
    background: #c73652;
    transform:  translateY(-2px);
}
a:hover   { text-decoration: underline; }

/* :focus — element focused ho (tab/click) */
input:focus {
    outline:        2px solid #00b4d8;
    outline-offset: 2px;
    border-color:   #00b4d8;
}

/* :active — click/tap hone par (button press) */
.btn:active {
    transform: scale(0.97);
    opacity:   0.9;
}
```

```css
/* :visited — already visit ki hui link */
a:visited { color: #9d4edd; }

/* :focus-within — andar koi element focused ho */
.form-group:focus-within {
    border-color: #06d6a0;
}

/* :focus-visible — sirf keyboard focus pe (mouse nahi) */
.btn:focus-visible {
    outline: 3px solid #ffd166;
}
```

Structural Pseudo-Classes — Position Based

structural-pseudo.css

```css
/* :first-child — parent ka pehla child */
li:first-child   { font-weight: 700;     }
p:first-child    { margin-top:  0;       }

/* :last-child — parent ka aakhri child */
li:last-child    { border-bottom: none;  }
p:last-child     { margin-bottom: 0;     }

/* :nth-child(n) — n number child */
li:nth-child(2)  { color: orange;        }  /* sirf doosra */
li:nth-child(odd) { background: #0d1b2a; }  /* 1,3,5,7... */
li:nth-child(even){ background: #0f2340; }  /* 2,4,6,8... */
li:nth-child(3n) { color: #e94560;      }  /* 3,6,9,12... */
li:nth-child(3n+1){ color: #06d6a0;     }  /* 1,4,7,10... */

/* :nth-last-child(n) — end se count karo */
li:nth-last-child(1) { color: gold;  }  /* last element */
```

```css
/* :only-child — parent ka ek hi child ho */
p:only-child { text-align: center; }

/* :first-of-type / :last-of-type — type ke hisaab se */
p:first-of-type  { font-size: 1.1rem;  }
p:last-of-type   { margin-bottom: 0; }
h2:nth-of-type(2){ color: #ffd166;    }
```

Form & State Pseudo-Classes

form-pseudo.css

```css
input:checked   { accent-color: #06d6a0;    }  /* checked checkbox/radio */
input:disabled  { opacity:      0.5;          }  /* disabled input */
input:enabled   { cursor:       text;         }  /* enabled input */
input:required  { border-color: #e94560;     }  /* required field */
input:optional  { border-color: #1e3a5f;     }  /* optional field */
input:valid     { border-color: #06d6a0;     }  /* valid value */
input:invalid   { border-color: #e94560;     }  /* invalid value */
input:read-only { background:   #0a0f1e;     }  /* readonly input */
input:placeholder-shown {
    border-color: #1e3a5f;   /* jab placeholder dikh raha ho */
}
```

Other Useful Pseudo-Classes

other-pseudo.css

```css
/* :not() — yeh nahi wale select karo */
p:not(.special)      { color:  #a8b2d8; }  /* special class ke alawa saare p */
li:not(:last-child)  { border-bottom: 1px solid #1e3a5f; }
input:not([type="submit"]) { width: 100%; }
```

```css
/* :empty — koi content nahi jis element mein */
div:empty { display: none; }

/* :root — document ka root element (html) */
:root {
    --primary:   #0f3460;
    --accent:    #e94560;
    --font-size: 16px;
}
/* :root mein CSS variables define karo — :root body se zyada specific hai */

/* :has() — parent selector (agar child ho) — Modern CSS! */
.card:has(img) {
    padding-top: 0;   /* card jiske andar img ho */
}
form:has(input:invalid) {
    border-color: red;  /* form jisme koi invalid input ho */
}
```

■ **Note:** `:has()` ek game-changer hai — pehli baar CSS mein parent ko child ke basis pe style kar sakte hain! Chrome 105+, Safari 15.4+ mein support hai.

// TOPIC 05 — ADVANCED

Pseudo-Elements

Element ke specific part ko style karo — :: (double colon)

KYA HAI?

Pseudo-elements se element ke **specific part** ko style karte hain — jaise pehla letter, pehli line, ya element ke before/after content. Double colon `::` se likhte hain. `::before` aur `::after` sabse zyada use hote hain — inse HTML mein extra element daaley bina design elements bana sakte ho!

::before aur ::after — Sabse Important!

before-after.css

```css
/* ::before — element ke content se PEHLE insert hota hai */
/* ::after  — element ke content ke BAAD insert hota hai  */
/* ZARURI: content property hona chahiye — "" bhi chalega */

/* ── Example 1: Quote marks ── */
blockquote::before {
    content:     '"';
    font-size:   4rem;
    color:       #e94560;
    line-height: 0;
}

/* ── Example 2: Required field star ── */
label.required::after {
    content: ' *';
    color:   #e94560;
}
```

```css
/* ── Example 3: Tooltip ── */
[data-tooltip]::after {
    content:    attr(data-tooltip);  /* attribute ki value use karo! */
    position:   absolute;
    background: #0f2340;
    padding:    4px 10px;
    border-radius: 4px;
    font-size:  12px;
    display:    none;
}
[data-tooltip]:hover::after {
    display: block;
}

/* ── Example 4: Decorative line (underline effect) ── */
h2 { position: relative; display: inline-block; }
h2::after {
    content:    '';             /* empty content — sirf design element */
    position:   absolute;
    bottom:     -4px;
    left:       0;
    width:      100%;
    height:     3px;
    background: #e94560;
}

/* ── Example 5: Clearfix (float issue fix) ── */
.clearfix::after {
    content: '';
    display: block;
    clear:   both;
}
```

Baaki Pseudo-Elements

other-pseudo-elements.css

```css
/* ::first-letter — pehla letter */
p::first-letter {
    font-size:   3em;
    float:       left;
    color:       #e94560;
    line-height: 1;
    margin-right:8px;    /* Drop cap effect — newspaper style! */
}

/* ::first-line — pehli line */
p::first-line {
    font-weight: 600;
    color:       white;
}

/* ::selection — user ne text select kiya tab */
::selection {
    background: #e94560;
    color:      white;
}

/* ::placeholder — input ka placeholder text */
input::placeholder {
    color:      #4a6080;
    font-style: italic;
}

/* ::marker — list item ka bullet/number */
li::marker {
    color:     #e94560;
    font-size: 1.2em;
```

```
}

/* ::backdrop — fullscreen element ke peeche */
dialog::backdrop {
    background: rgba(0,0,0,0.7);
}
```

Pseudo-Element	Kya Karta Hai
`::before`	Element ke content se pehle kuch insert karo (content: zaroori)
`::after`	Element ke content ke baad kuch insert karo (content: zaroori)
`::first-letter`	Paragraph ka pehla letter — drop cap effect
`::first-line`	Paragraph ki pehli line
`::selection`	User-selected text ka style
`::placeholder`	Input placeholder text ka style
`::marker`	List bullet ya number ka style
`::backdrop`	Fullscreen/dialog ke peeche overlay

■ **Remember:** `::before` aur `::after` mein `content: ""` hamesha likhna zaroori hai — warna yeh render hi nahi hoga! Chahe content empty ho.

// TOPIC 06 — ADVANCED

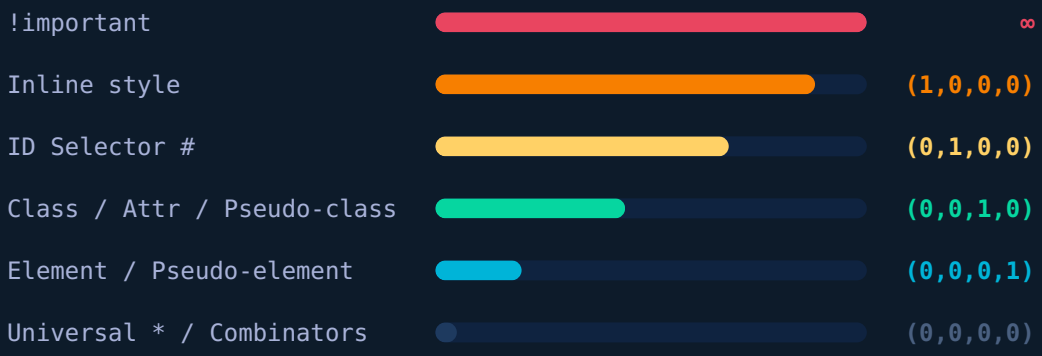
CSS Specificity — Priority System

Jab multiple rules clash karein — kaun jeete ga?

KYA HAI?

Specificity ek **weight system** hai jo decide karta hai ki jab multiple CSS rules ek hi element pe apply hoti hain, toh kaunsi rule ka style final mein dikhega. Har selector type ki ek specificity value hoti hai — **jyada specific selector jeeta hai!**

Specificity Points — (A, B, C) System



specificity-examples.css

```
/* Specificity calculate karo: (ID, Class, Element) */

p               { color: blue;   } /* (0,0,1) — 1 element */
.text           { color: green;  } /* (0,1,0) — 1 class */
#para           { color: red;    } /* (1,0,0) — 1 ID */

/* Complex selectors ka calculation: */
```

```
div p           { color: blue;   }  /* (0,0,2) — 2 elements */
.card p         { color: green;  }  /* (0,1,1) — 1 class + 1 element */
#nav .link      { color: yellow; }  /* (1,1,0) — 1 ID + 1 class */
#nav .link span { color: pink;   }  /* (1,1,1) — 1 ID + 1 class + 1 element */

/* Inline style — sabse zyada (ID se bhi zyada!) */
<p style="color: orange;">  /* (1,0,0,0) */

/* !important — sab override karta hai */
p { color: purple !important; }  /* inline ko bhi override karega! */

/* Tie hone par — baad wala rule jeeta hai! */
p { color: blue;  }
p { color: green; }  /* ← green dikhega — baad mein aaya hai */
```

■ **Pro Tip:** Specificity calculate karne ke liye **specificity.keegan.st** website use karo — selector paste karo aur instantly score milega! Debugging ke liye bahut helpful hai.

■ **Best Practice:** `!important` se bachne ki koshish karo — ek baar use karo toh doosri jagah `!important` likhna padega override karne ke liye. Maintenance nightmare ban jaata hai!

// TOPIC 07 — ADVANCED MODERN CSS

:is(), :where(), :not() — Modern CSS

Code reduce karo aur smarter selectors likho

KYA HAI?

Yeh teen pseudo-classes modern CSS mein bahut useful hain — inse **complex selectors ko simplified** likha ja sakta hai. Code kam hota hai aur readability better hoti hai. Sab modern browsers mein support hai.

:is() — Match Any

is-selector.css

```css
/* :is() — grouped matching, code reduce karta hai */

/* PURANA TARIKA — repetitive */
header h1, header h2, header h3,
main h1,   main h2,   main h3,
footer h1, footer h2, footer h3 {
    color: white;
}

/* NAYA TARIKA — :is() se simple! */
:is(header, main, footer) :is(h1, h2, h3) {
    color: white;
}

/* :is() ki specificity = list mein sabse specific selector */
:is(#nav, .link, p) {   /* specificity = ID ki = (1,0,0) */
```

```
    color: white;
}
```

:where() — Zero Specificity

where-selector.css

```
/* :where() — :is() jaisa lekin specificity ZERO hoti hai! */
/* Override karna aasaan ho jaata hai */

/* CSS Reset ke liye perfect — easily override ho sake */
:where(h1, h2, h3, h4, h5, h6) {
    font-weight: 700;
    line-height: 1.2;
}
/* Specificity = (0,0,0) — koi bhi rule override kar sakta hai */

/* Component libraries mein use hota hai */
:where(.btn, .link, .tag) {
    cursor:          pointer;
    text-decoration: none;
}
/* User apna .btn style easily override kar sakta hai */
```

:not() — Exclusion Selector

not-selector.css

```
/* :not() — yeh wale CHHOD ke baaki sab */

p:not(.intro)          { color: #a8b2d8;  }
li:not(:last-child)    { border-bottom: 1px solid #1e3a5f; }
input:not([disabled]) { background:  #0d1b2a;  }
div:not(.card):not(.hero) {
```

```
    padding: 10px;    /* card aur hero ke alawa saare divs */
}

/* Modern :not() — multiple values (CSS Selectors Level 4) */
p:not(.intro, .outro) {
    color: #a8b2d8;    /* intro aur outro dono chhod do */
}
```

Selector	Kya Karta Hai	Specificity
`:is(A, B, C)`	A ya B ya C — match karo	Sabse specific wala
`:where(A, B, C)`	:is() jaisa lekin override aasaan	Hamesha 0,0,0
`:not(A)`	A ke alawa sab	Argument ki specificity
`:has(A)`	Jiske andar A ho (parent selector)	Argument ki specificity

■ **Tip:** `:where()` CSS resets aur base styles ke liye perfect hai — specificity zero hone ki wajah se developer apni custom styles easily override kar sakta hai bina `!important` ke!

// QUICK REFERENCE

CSS Selectors — Complete Summary

Basic se Advanced — Saare selectors ek jagah

Category	Selector	Example	Kya Select Karta Hai
Basic	*	*	Saare elements
Basic	element	p, h1	Us type ke saare tags
Basic	.class	.btn	Us class wale elements
Basic	#id	#navbar	Us ID wala ek element
Basic	A, B	h1, h2	A aur B dono
Combinator	A B	nav a	A ke andar ke saare B
Combinator	A > B	ul > li	A ke direct children B
Combinator	A + B	h2 + p	A ke theek baad wala B
Combinator	A ~ B	h2 ~ p	A ke baad ke saare B
Attribute	[attr]	[required]	Attribute exist kare
Attribute	[attr="v"]	[type="email"]	Exact match

Category	Selector	Example	Kya Select Karta Hai
Attribute	[attr^="v"]	[href^="https"]	Se shuru ho
Attribute	[attr$="v"]	[href$=".pdf"]	Pe khatam ho
Attribute	[attr*="v"]	[href*="youtube"]	Kahin bhi ho
Pseudo-class	:hover	a:hover	Mouse upar ho
Pseudo-class	:focus	input:focus	Element focused ho
Pseudo-class	:nth-child(n)	li:nth-child(odd)	nth position wala
Pseudo-class	:not()	p:not(.intro)	Yeh chhod ke baaki
Pseudo-class	:has()	.card:has(img)	Jiske andar yeh ho
Pseudo-element	::before	h2::before	Content se pehle
Pseudo-element	::after	h2::after	Content ke baad
Pseudo-element	::placeholder	input::placeholder	Placeholder text
Pseudo-element	::selection	::selection	Selected text
Modern	:is()	:is(h1,h2,h3)	Koi bhi match kare
Modern	:where()	:where(h1,h2)	:is() + zero specificity

▸ NEXT TOPICS TO COVER

Colors in CSS

Fonts & Typography

CSS Units (px, rem, em, %)

Display Property

Flexbox

CSS Grid

Positioning

CSS Animations

Made with ♥ for Web Development Course learners

Shauray ke saath CSS seekhte raho 🚀 | Topic 03 — CSS Selectors